

Estratégias para lidar com chamadas bloqueantes

Redes de Computadores - Irineu Sotoma

Objetivo

Observar, na prática, cinco estratégias clássicas para tratar chamadas bloqueantes em programação com sockets: 1. múltiplas threads; 2. modo não bloqueante; 3. multiplexação com `select()`; 4. timeout; 5. opções de socket como `SO_REUSEADDR` e `SO_BROADCAST`.

Pré-requisitos

Python 3.10+ instalado.

Um terminal por servidor e, idealmente, outro(s) terminal(is) para cliente(s).

Ambiente Linux, macOS ou Windows (com pequenas diferenças no caso do exemplo com `setsockopt()`).

Arquivos do laboratório

Servidores

`server_multithread.py`

`server_select.py`

`server_timeout.py`

`server_setsockopt.py`

`server_reuseaddr.py`

`server_broadcast.py`

Clientes

`client_multithread.py`

`client_select.py`

`client_timeout.py`

`client_setsockopt.py`

`client_reuseaddr.py`

`client_broadcast.py`

Experimento 1 — Servidor multithread

Objetivo

Verificar que um servidor pode atender vários clientes simultaneamente criando uma thread por conexão.

Passos

1. Abra um terminal e execute: `python3 server_multithread.py`
2. Abra dois ou mais terminais adicionais.
3. Em cada terminal, execute: `python3 client_multithread.py`

O que observar

O servidor cria uma thread para cada cliente.

Uma thread pode ficar bloqueada em `recv()` enquanto as demais continuam funcionando.

As mensagens retornam em maiúsculas.

Perguntas

O que acontece no servidor quando um cliente fecha a conexão?

O que muda se vários clientes enviarem mensagens ao mesmo tempo?

Experimento 2 — Multiplexação com select()

Objetivo

Verificar que um único processo pode monitorar vários sockets ao mesmo tempo.

Passos

1. Execute o servidor: `python3 server_select.py`
2. Em dois ou mais terminais, execute: `python3 client_select.py`

O que observar

O servidor não cria threads por cliente.

O `select()` retorna quando há:

- nova conexão no socket de escuta;
- dados disponíveis em sockets de clientes.

O servidor atende vários clientes dentro de um único loop.

Perguntas

Qual a diferença conceitual entre esse servidor e o multithread?

Em que situação `select()` pode ser mais interessante que uma thread por cliente?

Experimento 3 — Timeout com `settimeout()`

Objetivo

Mostrar que a aplicação pode limitar o tempo máximo de espera de uma operação bloqueante.

Passos

1. Execute o servidor: `python3 server_timeout.py`
2. Em outro terminal, execute: `python3 client_timeout.py`

O que observar

O servidor dorme 5 segundos antes de responder.

O cliente espera apenas 3 segundos por causa de `settimeout(3.0)`.

O resultado esperado é um timeout no `recv()`.

Perguntas

O timeout significa que a conexão caiu?

Qual a diferença entre timeout no `connect()` e timeout no `recv()`?

Experimento 4 — Timeout com setsockopt()

Objetivo

Ver que o próprio socket pode receber opções de timeout no nível do sistema operacional.

Passos

1. Execute o servidor: `python3 server_setsockopt.py`
2. Em outro terminal, execute: `python3 client_setsockopt.py`

O que observar

O servidor também atrasa a resposta por 5 segundos.

O cliente configurou `SO_RCVTIMEO` e `SO_SNDTIMEO`.

Dependendo do sistema, o erro aparecerá como `OSError`.

Perguntas

Em Python, o que é mais simples: `settimeout()` ou `setsockopt()`?

Em quais cenários faria sentido usar `setsockopt()` diretamente?

Experimento 5 — Reuso de endereço com SO_REUSEADDR

Objetivo

Observar o efeito de `SO_REUSEADDR` em reinícios rápidos do servidor.

Passos

1. Execute o servidor: `python3 server_reuseaddr.py`
2. Em outro terminal, execute: `python3 client_reuseaddr.py`
3. Reinicie rapidamente o servidor e execute o cliente novamente.

O que observar

O servidor volta a fazer `bind()` rapidamente.

Em muitos cenários, sem `SO_REUSEADDR`, o reinício rápido poderia resultar em “Address already in use”.

Perguntas

Por que essa opção é tão comum em servidores de laboratório?

Ela muda alguma coisa do lado do cliente?

Experimento 6 — UDP broadcast com SO_BROADCAST

Objetivo

Mostrar o envio e recebimento de um datagrama em broadcast IPv4.

Passos

1. Execute o receptor: `python3 server_broadcast.py`
2. Em outro terminal, execute o emissor: `python3 client_broadcast.py`

O que observar

O receptor fica aguardando em `recvfrom()`.

O cliente envia um datagrama para `<broadcast>`.

O receptor exibe o conteúdo recebido e o endereço do remetente.

Perguntas

Por que esse exemplo usa UDP e não TCP?

Em que tipos de aplicação o broadcast pode ser útil?

Estratégias para lidar com chamadas bloqueantes
Redes de Computadores - Irineu Sotoma

Síntese final

Estratégias observadas

Threads: isolamento simples entre clientes.

select(): um único processo observa vários sockets.

Timeout: evita espera infinita.

SO_REUSEADDR: facilita reinício rápido do servidor.

SO_BROADCAST: permite envio UDP para broadcast.

Questão de fechamento

Qual dessas abordagens você escolheria para:

1. um servidor simples para poucos clientes;
2. um servidor com muitas conexões simultâneas;
3. um cliente que não pode esperar indefinidamente pela resposta?